

REGISTRO DE MOBILE - SEMANA 17

Resolução da Atividade: Entre relacional e NoSQL: qual escolher?

1. Escolha do Banco de Dados e Justificativa

Sistema de Controle Financeiro \(\rightarrow\) Banco de Dados Relacional (Ex: PostgreSQL / MySQL)

Justificativa: Transações financeiras exigem consistência absoluta. Os bancos relacionais utilizam o princípio **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade). Isso garante que, se uma transferência bancária falhar no meio do caminho, o dinheiro não suma: a operação é desfeita por inteiro, impedindo saldos incorretos.

Aplicativo de Rede Social \(\rightarrow\) Banco de Dados NoSQL (Ex: MongoDB para documentos / Redis para reações)

Justificativa: Redes sociais geram um volume gigantesco de dados variados (textos de tamanhos diferentes, fotos, vídeos, curtidas) a todo segundo. O NoSQL permite **escalabilidade horizontal** (adicionar mais servidores facilmente) e possui um esquema flexível, ideal para armazenar posts que mudam de formato constantemente.

2. Desafios de cada cenário

- **Desafio no Relacional (Financeiro):** Conforme o volume de transações históricas cresce para a casa dos milhões, consultas que cruzam muitas tabelas (*JOINS*) podem se tornar lentas, exigindo estratégias avançadas de indexação e otimização de hardware.
- **Desafio no NoSQL (Rede Social):** Manter a consistência imediata. Por exemplo, se um usuário atualizar a foto de perfil, pode demorar alguns milissegundos ou segundos para que essa alteração apareça para todos os seguidores (chamado de *consistência eventual*).

3. Exemplos de Operações (Modeladas no código abaixo)

- **Relacional:** Inserir uma transação garantindo que o saldo seja alterado e ler o extrato de forma exata.
- **NoSQL:** Criar um post dinâmico na rede social (um post com imagem e outro apenas com texto) sem precisar alterar a estrutura do banco.

4. Impacto no Longo Prazo (Desempenho e Manutenção)

- **Relacional:** A manutenção é facilitada pela rigidez dos dados (regras claras evitam dados corrompidos), mas o custo de desempenho e de servidores para escalar horizontalmente no futuro costuma ser bem mais alto.
- **NoSQL:** O desempenho se mantém veloz mesmo com bilhões de registros. Contudo, a manutenção exige mais atenção do time de desenvolvimento para garantir que a falta de um esquema rígido não resulte em dados bagunçados ou perdidos no código da aplicação.

Código teste (OneCompiler):

```
1 // SIMULAÇÃO DE BANCOS DE DADOS: RELACIONAL VS NOSQL
2
3 // =====
4 // 1. CENÁRIO RELACIONAL (Sistema Financeiro)
5 // =====
6 console.log("--- 1. SIMULANDO BANCO DE DADOS RELACIONAL (Tabelas Rígidas) ---")
7
8 // Estrutura fixa simulando tabelas com chaves primárias e estrangeiras
9 const tabelaUsuarios = [
10   { id_usuario: 1, nome: "Carlos Silva", saldo: 500.00 }
11 ];
12
13 const tabelaTransacoes = [];
14
15 function realizarTransacao(idUsuario, tipo, valor) {
16   const usuario = tabelaUsuarios.find(u => u.id_usuario === idUsuario);
17
18   if (!usuario) {
19     console.log("Erro: Usuário não encontrado. Transação cancelada.");
20     return;
21   }
22
23   // Garantindo a integridade dos dados (Simulação de transação ACID)
24   if (tipo === "Debito" && usuario.saldo < valor) {
25     console.log("⚠ Falha na transação: Saldo insuficiente para $usuario.nome")
26   }
27 }
28
29 --- 2. SIMULANDO BANCO NOSQL (Coleção de Documentos JSON Flexíveis) ---
30
31 Post salvo com sucesso no NoSQL!
32 Post salvo com sucesso no NoSQL!
33
34 Visualizando a coleção NoSQL (Esquema Flexível):
35 [
36   {
37     "_id": "dsfu86753",
38     "autor": "@dev_joao",
39     "conteudo": "Estudando bancos de dados hoje no OneCompiler!",
40     "hashtags": [
41       "#programacao",
42       "#nosql"
43     ],
44     "data_criacao": "2026-08-19T11:02:31.608Z"
45   },
46   {
47     "_id": "tuhni074h"
```